AI FOR HACKERS — COMPLETE BEGINNER'S GUIDE

From Zero to AI-Powered Security Professional

📚 STRUCTURED LEARNING PATH

PHASE 1: FOUNDATIONS (Weeks 1-2)

Goal: Understand AI basics and set up your environment

1.1 Introduction to AI for Hackers

- **What is AI?** Simulation of human intelligence by machines (learning, reasoning, self-correction)
- **The AI Hierarchy:**
  - **AI (Broad):** Game playing, decision systems
  - **Machine Learning (Focused):** Spam filtering, anomaly detection
  - **Deep Learning (Deep):** Image recognition, NLP
  - **Large Language Models (Current):** ChatGPT, code generation, hacking tools
- **Why Hackers Need AI:**
  - Automation at scale
  - Pattern recognition for vulnerabilities
  - Natural language power for phishing/social engineering
  - Evasion techniques (polymorphic code)
  - Efficiency (days of work → minutes)

1.2 Setting Up Your AI Hacking Lab Required Tools & Software:

Category	Tool/Software	Purpose
OS	Kali Linux / Ubuntu 22.04	Primary hacking OS
Language	Python 3.10+	AI/ML programming
AI Framework	PyTorch / TensorFlow	Deep learning
ML Library	Scikit-learn	Classical ML
NLP	Hugging Face Transformers	LLM interaction
Data	Pandas, NumPy	Data manipulation
Visualization	Matplotlib, Seaborn	Data visualization
API Access	OpenAI API / Anthropic API	LLM access
Notebooks	Jupyter Lab	Interactive development
Containers	Docker	Isolated environments
Version Control	Git + GitHub	Code management

Environment Setup Steps:

```
# Step 1: Install Python
sudo apt update && sudo apt install python3 python3-pip


# Step 2: Install virtualenv
```

```
pip3 install virtualenv

# Step 3: Create your AI hacking environment
virtualenv ai_hacking_env
source ai_hacking_env/bin/activate

# Step 4: Install core AI/ML libraries
pip install numpy pandas matplotlib seaborn scikit-learn
pip install torch torchvision torchaudio
pip install transformers datasets tokenizers
pip install openai anthropic jupyter
pip install requests beautifulsoup4 scapy
```

Verification Script:

```
import numpy as np
import pandas as pd
import sklearn
import torch
import transformers

print(f'NumPy: {np.__version__}')
print(f'Pandas: {pd.__version__}')
print(f'Scikit-learn: {sklearn.__version__}')
print(f'PyTorch: {torch.__version__}')
print(f'Transformers: {transformers.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
print('All libraries loaded successfully!')
```

PHASE 2: PYTHON & DATA FUNDAMENTALS (Weeks 3-4)

Goal: Master Python for AI/Security applications

2.1 Python Essentials for AI Security Data Structures You'll Use Daily:

```
# Lists - ordered, mutable sequences
targets = ['192.168.1.1', '192.168.1.2', '10.0.0.1']

# Dictionaries - key-value pairs (like JSON)
vuln = {
    'host': '192.168.1.1',
    'port': 22,
    'cve': 'CVE-2023-1234',
    'severity': 'HIGH',
    'exploitable': True
}

# List comprehensions - powerful one-liners
high_severity = [v for v in vulns if v['severity'] == 'HIGH']
```

```
# Sets - unique elements, fast lookup
open_ports = {22, 80, 443, 8080}
critical_ports = {22, 23, 3389}
exposed = open_ports.intersection(critical_ports)   # {22}
```

Functions, Classes & Decorators:

```
# Functions with type hints (good practice in AI code)
def preprocess_payload(raw: str, max_len: int = 512) -> list:
    tokens = raw.lower().split()
    return tokens[:max_len]

# Classes for organizing ML pipelines
class VulnClassifier:
    def __init__(self, model_path: str):
        self.model = self._load_model(model_path)
        self.threshold = 0.7

    def predict(self, features: list) -> str:
        score = self.model.predict_proba([features])[0][1]
        return 'VULNERABLE' if score > self.threshold else 'SAFE'

    def _load_model(self, path):
        import joblib
        return joblib.load(path)
```

2.2 NumPy & Pandas Essentials

```
import numpy as np
import pandas as pd

# NumPy arrays - the foundation of ML
features = np.array([[1, 0, 443, 1], [0, 1, 22, 0], [1, 1, 80, 1]])
labels = np.array([1, 0, 1])  # 1=attack, 0=normal

# Array operations (vectorized = fast)
normalized = (features - features.mean(axis=0)) / features.std(axis=0)

# Pandas DataFrames for structured data
df = pd.read_csv('network_logs.csv')
print(df.head())          # First 5 rows
print(df.describe())      # Statistical summary
print(df.isnull().sum()) # Check for missing values

# Feature engineering
df['is_suspicious'] = (df['bytes_sent'] > 10000) & (df['port'] == 22)
df['hour'] = pd.to_datetime(df['timestamp']).dt.hour

# Select features for model
X = df[['bytes_sent', 'bytes_recv', 'port', 'duration', 'hour']].values
y = df['label'].values
```

Key Concept: Feature Engineering Transform raw data into meaningful inputs for ML models. Good security features: packet size, connection duration, time of day, protocol type, byte ratios.

PHASE 3: MACHINE LEARNING FUNDAMENTALS (Weeks 5-6)

Goal: Understand ML types and build your first security classifier

3.1 Types of Machine Learning

Type	How It Works	Security Use Cases
Supervised	Learns from labeled examples	Malware classification, phishing detection
Unsupervised	Finds patterns without labels	Anomaly detection, network clustering
Reinforcement	Learns via rewards/penalties	Automated penetration testing, evasion
Semi-supervised	Mix of labeled & unlabeled	Threat intel with limited labels
Self-supervised	Labels from data itself	Log analysis, LLM pre-training

3.2 Supervised Learning Example: Network Intrusion Detection

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np

# Load the NSL-KDD intrusion detection dataset
df = pd.read_csv('nsl_kdd.csv')

# Feature selection
features = ['duration', 'src_bytes', 'dst_bytes', 'land', 'urgent',
            'hot', 'num_failed_logins', 'logged_in', 'num_compromised']
X = df[features].astype(float)
y = (df['label'] != 'normal').astype(int)  # 1=attack, 0=normal

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Random Forest
clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
clf.fit(X_train, y_train)

# Evaluate
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred,
                            target_names=['Normal', 'Attack']))
```

3.3 Model Evaluation & Metrics

Metric	Formula	When to Use
Accuracy	Correct/Total	Balanced classes only
Precision	TP / (TP + FP)	Minimize false alarms
Recall	TP / (TP + FN)	Minimize missed attacks
F1-Score	2*P*R/(P+R)	Balance precision/recall
AUC-ROC	Area under ROC curve	Threshold-independent eval

Security Focus: In intrusion detection, **HIGH RECALL** is usually more important than high precision. Missing a real attack (false negative) is more dangerous than a false alarm (false positive).

PHASE 4: NEURAL NETWORKS & DEEP LEARNING (Weeks 7-8)

Goal: Build neural networks for security applications

4.1 How Neural Networks Work

- **Input Layer:** Receives raw features (e.g., network packet features)
- **Hidden Layers:** Transform inputs through weighted connections and activation functions
- **Output Layer:** Produces final predictions (e.g., attack probability)

4.2 Building Your First Neural Network with PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Define a simple intrusion detection neural network
class IntrusionDetector(nn.Module):
    def __init__(self, input_dim: int):
        super(IntrusionDetector, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
```

```
            nn.Linear(32, 1),
            nn.Sigmoid()  # Output: attack probability
        )

    def forward(self, x):
        return self.network(x)

# Training loop
model = IntrusionDetector(input_dim=41)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.BCELoss()

for epoch in range(50):
    model.train()
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        predictions = model(batch_X).squeeze()
        loss = criterion(predictions, batch_y)
        loss.backward()
        optimizer.step()
```

4.3 Activation Functions

Function	Range	Best Used For
ReLU	[0, inf)	Hidden layers (default choice)
Sigmoid	(0, 1)	Binary classification output
Softmax	(0, 1) summing to 1	Multi-class classification output
Tanh	(-1, 1)	Hidden layers, NLP tasks
LeakyReLU	(-inf, inf)	Preventing dying ReLU problem

PHASE 5: LARGE LANGUAGE MODELS (LLMs) (Weeks 9-10)

Goal: Master LLMs for security research

5.1 How LLMs Work

- **Transformer Architecture** (2017): "Attention Is All You Need"
- **Key Components:**
  - Self-Attention: Weighs importance of each word relative to others
  - Multi-Head Attention: Multiple attention operations in parallel
  - Positional Encoding: Injects position information
  - Feed-Forward Networks: Non-linear transformations
  - Layer Normalization: Stabilizes training

5.2 Working with LLMs via API

```
import openai
```

```
client = openai.OpenAI(api_key='your-api-key-here')

# Basic completion for security analysis
def analyze_code_for_vulns(code_snippet: str) -> str:
    response = client.chat.completions.create(
        model='gpt-4',
        messages=[
            {
                'role': 'system',
                'content': 'You are a senior security researcher. '
                           'Analyze code for vulnerabilities, '
                           'explain the issue, and suggest fixes.'
            },
            {
                'role': 'user',
                'content': f'Analyze this code:

{code_snippet}'
            }
        ],
        max_tokens=1000
    )
    return response.choices[0].message.content

# Example usage
vuln_code = '''
def login(username, password):
    query = f"SELECT * FROM users WHERE user='{username}'"
    return db.execute(query)  # SQL injection!
'''
print(analyze_code_for_vulns(vuln_code))
```

5.3 Tokenization & Embeddings

```
from transformers import AutoTokenizer, AutoModel
import torch

# Use a security-focused model or general BERT
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')

def get_embedding(text: str) -> torch.Tensor:
    inputs = tokenizer(text, return_tensors='pt',
                       truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model(**inputs)
    # Use [CLS] token embedding as sentence representation
    return outputs.last_hidden_state[:, 0, :]

# Compare two payloads semantically
emb1 = get_embedding("SELECT * FROM users WHERE id=1")
```

```
emb2 = get_embedding("SELECT * FROM users WHERE id=1 OR 1=1--")
similarity = torch.cosine_similarity(emb1, emb2)
print(f'Semantic similarity: {similarity.item():.4f}')
```

PHASE 6: PROMPT ENGINEERING FOR HACKERS (Weeks 11-12)

Goal: Craft effective prompts for security tasks

6.1 Core Prompt Techniques

- **Zero-Shot Prompting:** Directly asking without examples
- **Few-Shot Prompting:** Providing 2-5 examples to guide output
- **Chain-of-Thought:** Asking model to "think step by step"
- **Role Prompting:** Assigning a persona ("You are a senior penetration tester…")
- **Structured Output:** Requesting JSON/XML for programmatic use
- **Iterative Refinement:** Building on previous outputs

6.2 Security-Focused Prompt Templates Template 1: Vulnerability Analysis

```
VULN_ANALYSIS_PROMPT = '''
You are a CREST-certified penetration tester with 10 years experience.
Analyze the following code/configuration for security vulnerabilities.

For each vulnerability found, provide:
1. Vulnerability name and CWE identifier
2. Severity (Critical/High/Medium/Low)
3. Explanation of the vulnerability
4. Proof-of-concept exploit (if applicable)
5. Remediation recommendation

Code to analyze:
{code}
'''
```

Template 2: CTF Problem Solving

```
CTF_PROMPT = '''
You are an expert CTF (Capture the Flag) player.
Category: {category}
Challenge description: {description}
Files/Data provided: {data}

Think through this step by step:
1. What type of challenge is this?
2. What are the key observations?
3. What techniques apply here?
4. Provide the solution approach.
'''
```

6.3 Prompt Injection Attacks Types of Prompt Injection:

- **Direct Injection:** User overwrites system prompt directly
- **Indirect Injection:** Malicious instructions in external content
- **Stored Injection:** Saved to database, triggers later
- **Multimodal Injection:** Hidden in images, audio, or documents

Defense Against Prompt Injection:

- Separate user input from system instructions
- Implement input validation and output filtering
- Use privilege separation (minimal LLM permissions)
- Monitor and log all LLM inputs/outputs
- Never pass raw user input directly into prompts with elevated privileges

---

PHASE 7: AI-POWERED RECONNAISSANCE (Weeks 13-14)

Goal: Automate OSINT and target analysis

7.1 AI Recon Agent

```
import openai
import requests
from bs4 import BeautifulSoup

class AIReconAgent:
    def __init__(self, api_key: str):
        self.client = openai.OpenAI(api_key=api_key)
        self.intel = []

    def scrape_target(self, url: str) -> tuple:
        resp = requests.get(url, timeout=10)
        soup = BeautifulSoup(resp.text, 'html.parser')
        # Extract text, links, emails, tech signatures
        text = soup.get_text()[:3000]
        links = [a['href'] for a in soup.find_all('a', href=True)]
        return text, links

    def analyze_with_ai(self, raw_data: str, target: str) -> str:
        prompt = f'''
        You are an OSINT analyst. Analyze this data from {target}.
        Identify: Technologies used, employee names/emails,
        potential vulnerabilities, interesting endpoints.
        Data: {raw_data[:2000]}
        '''
        response = self.client.chat.completions.create(
            model='gpt-4',
            messages=[{'role': 'user', 'content': prompt}]
```

```
        )
        return response.choices[0].message.content
```

7.2 AI-Driven Footprinting

- **DNS Intelligence:** AI analyzes DNS records and subdomains
- **Social Media Mining:** Scrape and analyze employee profiles
- **Job Posting Analysis:** Extract technology stack information
- **Certificate Transparency:** Parse CT logs for subdomain discovery
- **Shodan/Censys Analysis:** AI interprets exposed service banners

Tool Spotlight: theHarvester + AI Feed theHarvester output into an LLM to automatically prioritize targets, identify organizational patterns, and generate structured recon reports.

PHASE 8: VULNERABILITY DISCOVERY WITH AI (Weeks 15-16)

Goal: Automate code auditing and bug finding

8.1 AI Code Auditor

```
import openai
import ast
import os

class AICodeAuditor:
    VULNERABILITY_PATTERNS = [
        'SQL injection', 'XSS', 'command injection',
        'path traversal', 'SSRF', 'XXE', 'deserialization',
        'hardcoded credentials', 'weak cryptography', 'race condition'
    ]

    def __init__(self, api_key: str):
        self.client = openai.OpenAI(api_key=api_key)
        self.findings = []

    def audit_file(self, filepath: str) -> list:
        with open(filepath, 'r') as f:
            code = f.read()

        prompt = f'''
        Security code review task. Analyze this Python code.
        Find ALL security vulnerabilities. For each:
        - Line number(s)
        - Vulnerability type (CWE-XXX)
        - Severity: CRITICAL/HIGH/MEDIUM/LOW
        - Explanation
        - PoC exploit string
        - Fix

        Return as JSON array.
```

```
        Code: {code}
        '''
        response = self.client.chat.completions.create(
            model='gpt-4',
            messages=[{'role': 'user', 'content': prompt}]
        )
        return response.choices[0].message.content
```

8.2 AI-Powered Fuzzing

```
# AI-guided mutation fuzzing example
import random
import openai

def ai_mutate_payload(original: str, context: str, client) -> str:
    prompt = f'''
    You are a fuzzing expert. Given this input and context,
    generate 5 mutations likely to trigger security bugs.
    Focus on: boundary values, special chars, format strings,
    encoding tricks, length extremes.

    Original: {original}
    Context: {context}
    Return as JSON array of strings.
    '''
    resp = client.chat.completions.create(
        model='gpt-4',
        messages=[{'role': 'user', 'content': prompt}]
    )
    import json
    return  json.loads(resp.choices[0].message.content)

# Generate fuzzing payloads
payloads = ai_mutate_payload(
    original='admin@company.com',
    context='Email input field with SQL backend',
    client=client
)
```

PHASE 9: OFFENSIVE AI TECHNIQUES (Weeks 17-18)

Goal: Understand adversarial ML and AI-powered attacks

9.1 Adversarial Machine Learning

```
import torch
import torch.nn.functional as F


def fgsm_attack(model, image, label, epsilon=0.1):
```

```
    '''Fast Gradient Sign Method (FGSM) adversarial example'''
    image.requires_grad = True

    # Forward pass
    output = model(image)
    loss = F.cross_entropy(output, label)

    # Backward pass
    loss.backward()

    # Create adversarial example
    data_grad = image.grad.data
    perturbed = image + epsilon * data_grad.sign()
    perturbed = torch.clamp(perturbed, 0, 1)

    return perturbed

# Use against IDS/malware classifier
adversarial_sample = fgsm_attack(
    ids_model, network_features, true_label, epsilon=0.05
)
# This 'normal-looking' traffic may now bypass the IDS
```

9.2 AI-Powered Social Engineering (Defensive Awareness)

- **Spear Phishing Generation:** LLMs generate personalized phishing emails
- **Voice Cloning:** 3-30 seconds of audio to clone a target's voice
- **Deepfake Video:** AI-generated video for BEC scams
- **Chatbot Impersonation:** AI chatbots mimicking IT helpdesk staff

⚠️ **Defensive Purpose:** Understanding these techniques is essential for security awareness training, phishing simulation programs, and building effective email security controls.

PHASE 10: DEFENSIVE AI & DETECTION (Weeks 19-20)

Goal: Build AI-powered defense systems

10.1 Anomaly Detection with AI

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

class NetworkAnomalyDetector:
    def __init__(self, contamination=0.01):
        self.model = IsolationForest(
            contamination=contamination, # Expected % anomalies
            n_estimators=100,
            random_state=42
```

```
        )
        self.scaler = StandardScaler()

    def fit(self, normal_traffic: pd.DataFrame):
        '''Train on baseline normal traffic'''
        features = self._extract_features(normal_traffic)
        scaled = self.scaler.fit_transform(features)
        self.model.fit(scaled)
        print('Baseline model trained on normal traffic')

    def predict(self, traffic: pd.DataFrame) -> np.ndarray:
        features = self._extract_features(traffic)
        scaled = self.scaler.transform(features)
        # Returns -1 for anomalies, 1 for normal
        return self.model.predict(scaled)

    def _extract_features(self, df: pd.DataFrame) -> np.ndarray:
        return df[['bytes_in', 'bytes_out', 'duration',
                   'packet_count', 'unique_ips']].values
```

10.2 AI-Powered SIEM

- **Alert Triage:** ML models prioritize thousands of daily alerts (up to 90% reduction)
- **Behavioral Analysis:** UEBA baselines normal behavior and flags deviations
- **Threat Correlation:** Graph neural networks correlate events across systems
- **Automated Response:** AI-driven SOAR platforms execute playbooks automatically
- **Natural Language Query:** Analysts query in plain English

Key Tools: Defensive AI Stack

- Elastic SIEM + ML
- Splunk Enterprise Security
- Microsoft Sentinel
- Darktrace (Network AI)
- CrowdStrike Falcon (EDR AI)
- Vectra AI (NDR)
- IBM QRadar with Watson

PHASE 11: REAL-WORLD PROJECTS (Weeks 21-24)

Goal: Build practical AI security tools

11.1 Project 1: AI Recon Bot

```
#!/usr/bin/env python3
# AI Recon Bot - Educational Purpose Only
import subprocess
```

```python
import openai
import json
from dataclasses import dataclass

@dataclass
class ReconResult:
    target: str
    subdomains: list
    open_ports: dict
    technologies: list
    ai_analysis: str

class AIReconBot:
    def __init__(self, target: str, api_key: str):
        self.target = target
        self.client = openai.OpenAI(api_key=api_key)
        self.result = ReconResult(target, [], {}, [], '')

    def run_subfinder(self) -> list:
        '''Find subdomains using subfinder'''
        cmd = f'subfinder -d {self.target} -silent'
        result = subprocess.run(cmd.split(), capture_output=True)
        subs = result.stdout.decode().strip().split('
')
        self.result.subdomains = subs
        return subs

    def analyze_attack_surface(self) -> str:
        '''Use AI to analyze collected data'''
        prompt = f'''
        You are a senior penetration tester.
        Target: {self.target}
        Subdomains: {self.result.subdomains[:20]}
        Open ports: {self.result.open_ports}
        Technologies: {self.result.technologies}

        Provide a structured attack surface analysis:
        1. Highest value targets
        2. Most likely attack vectors
        3. Recommended next steps
        '''
        resp = self.client.chat.completions.create(
            model='gpt-4',
            messages=[{'role': 'user', 'content': prompt}]
        )
        return resp.choices[0].message.content
```

11.2 Project 2: AI-Powered Password Cracker Assistant

```python
def generate_targeted_wordlist(target_info: dict, client) -> list:
    '''Generate smart wordlist using AI + OSINT data'''
```

```
    prompt = f'''
    Create a targeted password wordlist for a penetration test.
    Target info: {json.dumps(target_info)}

    Generate 50 likely passwords based on:
    - Company name variations
    - Employee names + common patterns
    - Location + year combinations
    - Industry-specific terms
    - Common patterns: Name123!, Company2024

    Return as JSON array of strings.
    '''
    resp = client.chat.completions.create(
        model='gpt-4',
        messages=[{'role': 'user', 'content': prompt}]
    )
    return  json.loads(resp.choices[0].message.content)

# Example
target = {
    'company': 'AcmeCorp',
    'city': 'London',
    'employees': ['John Smith', 'Alice Brown'],
    'founded': 2015
}
wordlist = generate_targeted_wordlist(target, client)
```

11.3 CTF Tips: Using AI in Competitions

- Feed challenge descriptions and files to GPT-4 for initial analysis
- Use AI to decode unknown encodings and formats
- Ask AI to explain assembly code and reverse engineering problems
- Use AI for pattern recognition in crypto challenges
- Build an AI-powered hint system that guides without giving answers

---

PHASE 12: ETHICS, LEGALITY & RESPONSIBLE DISCLOSURE (Week 25)

Goal: Understand legal boundaries and ethical responsibilities

12.1 Legal Boundaries Key Laws You Must Know:

- Computer Fraud and Abuse Act (CFAA) - USA
- Computer Misuse Act 1990 - UK
- EU Directive 2013/40/EU - Europe
- IT Act 2000 - India
- Cybercrime Act - Australia

The Rule of Written Authorization: Before testing ANY system, you must have explicit written authorization specifying:

- Exact IP ranges and domains in scope
- Types of testing permitted (scanning, exploitation, social engineering)
- Testing window (dates and times)
- Emergency contact procedures
- Data handling requirements
- Reporting requirements

12.2 Responsible Disclosure

1. **Document:** Thoroughly document vulnerability with reproduction steps
2. **Contact:** Find security contact (security.txt, HackerOne, Bugcrowd)
3. **Report Privately:** Send detailed, professional report with remediation suggestions
4. **Set Timeline:** Give vendor 90 days to fix before public disclosure
5. **Follow Up:** Check in at 30/60/90 days if no response
6. **Coordinate:** Work with vendor on patch timeline and CVE assignment
7. **Publish:** After patch release, publish technical writeup

12.3 AI Ethics in Security

- AI-generated exploits can proliferate attack capabilities to less skilled threat actors
- Automated AI attacks can cause unintended collateral damage at scale
- AI-powered surveillance tools raise serious privacy concerns
- Bias in AI security models can lead to discriminatory outcomes
- Always consider the dual-use nature of techniques you develop

---

🛠️ COMPLETE TOOLS ARSENAL

Offensive AI Tools

Tool	Purpose	Category
WormGPT	Uncensored LLM for phishing/ malware generation	Threat Actor Tool
FraudGPT	BEC content generation, fake invoices	Threat Actor Tool
PassGAN	GAN-based password generation from breach data	Password Cracking
DeepExploit	Automated pentesting using deep RL	Automated Exploitation
AutoAttacker	Automated spear-phishing with web search	Social Engineering
Ghauri	SQL injection with ML-based payload mutation	Web App Testing
Neural Fuzzer	LSTM-based intelligent fuzzing payloads	Fuzzing

Defensive AI Tools

Tool	Purpose	Category
Darktrace	Unsupervised ML for novel threat detection	NDR
Vectra AI	AI-powered network detection and response	NDR
Cylance (BlackBerry)	ML-based endpoint protection	EDR
SparkCognition	Deep learning for ICS/SCADA security	OT Security
Securonix	Next-gen SIEM with ML for UEBA	SIEM
ExtraHop Reveal(x)	Cloud-native NDR with ML	NDR
Recorded Future	AI-powered threat intelligence	Threat Intel

LLM Security Research Tools

```python
# Garak - LLM Vulnerability Scanner
# pip install garak
# Run vulnerability probes against any LLM

import garak
from garak.harnesses.probewise import ProbewiseHarness

# Configure the scan
harness = ProbewiseHarness()
harness.run(
    model='openai',
    model_name='gpt-3.5-turbo',
    probes=['promptinjection', 'jailbreak', 'dan', 'knownbadsignatures'],
    detectors=['always.Fail', 'productkey.ProductKey']
)

# LLMFuzzer - Automated jailbreak fuzzing
# Mutates and evolves prompts to find bypasses

from llmfuzzer import LLMFuzzer

fuzzer = LLMFuzzer(
    target_llm='gpt-4',
    attack_dataset='jailbreaks_v3.jsonl',
    mutation_strategy='genetic',
    generations=100
)
results = fuzzer.run()
print(f'Successful bypasses: {len(results.successful)}')
```

📖 REFERENCE LINKS & RESOURCES

Learning Platforms

- TryHackMe - Interactive cybersecurity training
- HackTheBox - Penetration testing labs
- PortSwigger Web Academy - Web security training
- fast.ai - Free deep learning course
- Coursera ML Specialization - Machine learning courses

AI Security Tools Repositories

- Adversarial Robustness Toolbox (IBM)
- CleverHans - Adversarial examples
- TextAttack - NLP attacks
- Garak - LLM vulnerability scanner
- LLMFuzzer - LLM fuzzing

CTF Platforms

- PicoCTF
- CTFtime.org
- NahamCon CTF
- DEF CON CTF Qualifier
- Google CTF

Communities & Conferences

- DEF CON AI Village
- r/netsec + r/MachineLearning
- OWASP AI Security Project
- Black Hat AI Summit

Certifications to Pursue

Certification	Description	Focus Area
OSCP	Offensive Security Certified Professional	Penetration Testing
CEH	Certified Ethical Hacker	AI-powered hacking concepts
GCIH/GCIA	GIAC Incident Handler/Intrusion Analyst	Defensive Focus
AWS/Azure/GCP ML	Cloud provider ML certifications	AI Deployment
MLOps Professional	MLOps certifications	Securing AI Pipelines

---

🚀 ADVANCED LEARNING PATHS

Path	Focus Area	Timeline
ML Engineer	Deep learning, model development, research	6-12 months
AI Red Teamer	Offensive AI, LLM security, adversarial ML	3-6 months
AI Blue Teamer	Defensive AI, anomaly detection, SIEM/SOAR	3-6 months
AI Security Researcher	Novel vulnerabilities, publications, CVEs	12-24 months
AI Product Security	Securing AI systems, SDLC, compliance	6-12 months

---

📋 QUICK REFERENCE CHEAT SHEETS

Common ML Algorithms for Security

Algorithm	Type	Best For	Complexity
Random Forest	Supervised	Malware classification, IDS	Low
Gradient Boosting	Supervised	Fraud detection, scoring	Medium
SVM	Supervised	Binary classification	Low-Med
K-Means	Unsupervised	Network segmentation, clustering	Low
DBSCAN	Unsupervised	Anomaly detection, botnet C2	Low
Isolation Forest	Unsupervised	Outlier/anomaly detection	Low
Autoencoders	Unsupervised	Network anomaly, malware	High
LSTM/GRU	Supervised	Log analysis, sequence attacks	High
Transformer	Both	NLP security, code analysis	Very High
GAN	Generative	Adversarial examples, data aug	Very High

OWASP Top 10 for LLM Applications

ID	Name	Description
LLM01	Prompt Injection	Attacker manipulates LLM via crafted inputs
LLM02	Insecure Output Handling	LLM output not validated before processing
LLM03	Training Data Poisoning	Manipulation of training data to introduce backdoors

Table 10 – continued

ID	Name	Description
LLM04	Model Denial of Service	Resource exhaustion through complex prompts
LLM05	Supply Chain Vulnerabilities	Compromised models, plugins, or data sources
LLM06	Sensitive Information Disclosure	LLM reveals PII or confidential training data
LLM07	Insecure Plugin Design	Plugins lack proper access controls
LLM08	Excessive Agency	LLM given too much power to act autonomously
LLM09	Overreliance	Trusting LLM output without verification
LLM10	Model Theft	Unauthorized access to proprietary models

Essential Python One-Liners for AI Security

```python
# Encode/decode base64
import base64; encoded = base64.b64encode(b'data'); decoded = base64.b64decode(encoded)

# Hash a string (MD5, SHA256)
import hashlib; hashlib.sha256(b'password').hexdigest()

# Generate secure random token
import secrets; secrets.token_hex(32)

# Quick HTTP request
import requests; r = requests.get('https://api.example.com')

# Parse JSON
import json; data = json.loads('{"key": "value"}')

# Extract emails from text
import re; emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', text)

# Check if port is open
import socket; s = socket.socket(); s.connect_ex(('target.com', 80))

# Simple word tokenization
import nltk; tokens = nltk.word_tokenize(text)

# Cosine similarity between vectors
from numpy import dot; from numpy.linalg import norm; cos_sim = dot(a, b) / (norm(a) * norm(b))
```

⚠️ CRITICAL SECURITY WARNINGS

Cloud AI Security Misconfigurations

Risk	Description	Severity
Open S3 Buckets	Training datasets and model artifacts exposed publicly	**CRITICAL**
Overprivileged IAM	ML services with admin permissions beyond training scope	**HIGH**
Unencrypted Models	Model artifacts stored without encryption at rest	**HIGH**
Public Endpoints	Model inference endpoints accessible without authentication	**CRITICAL**
Insecure Notebooks	Jupyter notebooks exposed without authorization	**HIGH**
No VPC Isolation	ML infrastructure not isolated in private subnets	**MEDIUM**
Training Data SSRF	ML data loaders fetching URLs from user-controlled input	**HIGH**
Dependency Confusion	Private ML packages overridden by malicious public ones	**HIGH**

Critical: Pickle Deserialization

PyTorch models saved with `torch.save()` use Python's pickle format by default. **Never load untrusted model files** — they can execute arbitrary code on deserialization.

Safe Loading:

```
# Use weights_only=True when loading from untrusted sources
torch.load('model.pth', weights_only=True)


# Always verify model checksums from trusted sources
```

🎯 FINAL WORDS

> "The most powerful hacking tool is your curiosity and creativity. AI amplifies your abilities, but it doesn't replace your judgment, ethics, or expertise."

Remember:

- Always hack legally
- Disclose responsibly
- Share knowledge with the community
- The security world is better when we work together

Keep learning. Keep hacking (legally).

---

created by Bhuvan